Capítulo 1: Introdução à Engenharia de Software

SOMMERVILLE, I. Engenharia de software. 9 ed. tradução Ivan Bosnic e Kalinka G.de O. Gonçalves ; revisão técnica Kechi Hirama. São Paulo : Pearson Prentice Hall, 2011. (Disponível na Biblioteca da UFGD)

1 Introdução

Os sistemas de software são abstratos e intangíveis.

Existem vários tipos de sistemas de software, desde os simples sistemas embutidos até os sistemas de informações complexos, de alcance mundial.

Todas essas aplicações precisam de engenharia de software, embora não necessitem das mesmas técnicas.

1.1 Desenvolvimento profissional de software

O software profissional, o que é usado por alguém além do seu desenvolvedor, é normalmente criado por equipes, em vez de indivíduos.

Ele é mantido e alterado durante sua vida.

A engenharia de software tem por objetivo apoiar o desenvolvimento profissional de software, mais do que a programação individual.

Engenheiros de software se preocupam em desenvolver produtos de software.

1. Produtos genéricos.
2. Produtos sob encomenda

1.1.1 Engenharia de software

Engenharia de software é uma disciplina de engenharia cujo foco está em todos os aspectos da produção de software, desde os estágios iniciais da especificação do sistema até sua manutenção, quando o sistema já está sendo usado. Há duas expressões importantes nessa definição:

1. Disciplina de engenharia. Engenheiros fazem as coisas funcionarem. Eles aplicam teorias, métodos e ferramentas onde for apropriado. No entanto, eles os usam seletivamente e sempre tentam descobrir as soluções para os problemas, mesmo quando não há teorias e métodos aplicáveis. Os engenheiros também reconhecem que devem trabalhar de acordo com as restrições organizacionais e financeiras, então buscam soluções dentro dessas restrições.

2. Todos os aspectos da produção de software. A engenharia de software não se preocupa apenas com os processos técnicos do desenvolvimento de software. Ela também inclui atividades como gerenciamento de projeto de software e desenvolvimento de ferramentas, métodos e teorias para apoiar a produção de software.

Engenharia de software é importante por dois motivos:

1. Cada vez mais, indivíduos e sociedades dependem dos sistemas de software avançados. Temos de ser capazes de produzir sistemas confiáveis econômica e rapidamente.

2. Geralmente é mais barato, a longo prazo, usar métodos e técnicas da engenharia de software para sistemas de software, em vez de simplesmente escrever os programas como se fossem algum projeto pessoal. Para a maioria dos sistemas, a maior parte do custo é mudar o software depois que ele começa a ser usado.

A abordagem sistemática usada na engenharia de software é, às vezes, chamada processo de software.

1. Especificação de software

2. Desenvolvimento de software,

2. Desenvolvimento de software,

4. Evolução de software,

… há três aspectos gerais que afetam vários tipos diferentes de software:

1. Heterogeneidade.

2. Mudança de negócio e social.

3. Segurança e confiança.

1.1.2 Diversidade na engenharia de software

Não existem técnicas e métodos universais na engenharia de software adequados a todos os sistemas e todas as empresas.

Em vez disso, um conjunto diverso de métodos e ferramentas de engenharia de software tem evoluído nos últimos 50 anos.

Existem muitos tipos diferentes de aplicações, incluindo:

1. Aplicações stand-alone.
2. Aplicações interativas baseadas em transações.
3. Sistemas de controle embutidos.
4. Sistemas de processamento de lotes.
5. Sistemas de entretenimento.
6. Sistemas para modelagem e simulação.
7. Sistemas de coleta de dados.
8. Sistemas de sistemas.

1.1.3 Engenharia de software e a Internet

O desenvolvimento da Internet teve efeito profundo em nossas vidas.

Web services são componentes de software acessados pela Internet e fornecem uma funcionalidade específica e útil.

1.2 Ética na engenharia de software

Assim como outras disciplinas de engenharia, a engenharia de software é desenvolvida dentro de um framework social e legal que limita a liberdade das pessoas que trabalham nessa área.

Organizações como ACM, IEEE (Institute of Electrical and Electronic Engineers) e British Computer Society publicam um código de conduta profissional ou código de ética.

1.3 Estudo de caso

Um estudo de caso representa a descrição de um problema real, de forma fictícia, para o estudo de possíveis soluções. Aplicando a teoria para realizar a análise do problema, formulação de estratégias de solução, tomada de decisão para escolher a solução, projetar e implementar a solução.

Os três tipos de sistema que uso como estudos de caso são:

1. Um sistema embutido.
2. Um sistema de informação.
3. Um sistema de coleta de dados baseado em sensores.

Capítulo 1: Introdução à Engenharia de Software

SOMMERVILLE, I. Engenharia de software. 9 ed. tradução Ivan Bosnic e Kalinka G.de O. Gonçalves; revisão técnica Kechi Hirama. São Paulo : Pearson Prentice Hall, 2011. (Disponível na Biblioteca da UFGD)

⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙

Data: 25/02/2025

Atividade em grupo.

Orientação:

1. Formem grupos de 3 integrantes;
2. Leiam a Tabela 1.1 Perguntas frequentes sobre software;
3. Discutam entre vocês as perguntas e respostas.
4. Para cada pergunta, em consenso:
    1. Vocês concordam com a resposta?
    2. Acrescentariam alguma(s) informação (ões)?
5. Registre as considerações de vocês em uma folha.
6. Apresentação e discussão do que foi discutido;

⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙⚙

Capítulo 1: Introdução à Engenharia de Software

SOMMERVILLE, I. Engenharia de software. 9 ed. tradução Ivan Bosnic e Kalinka G.de O. Gonçalves; revisão técnica Kechi Hirama. São Paulo : Pearson Prentice Hall, 2011. (Disponível na Biblioteca da UFGD)

⚙⚙⚙

DESENVOLVIMENTO:

- a partir do zero; ou
- a partir legado/ integração de prateleira ou componentes.

Existem muitos processos de software diferentes, mas todos devem incluir quatro atividades fundamentais para a engenharia de software:

1. Especificação de software. A funcionalidade do software e as restrições a seu funcionamento devem ser definidas.

2. Projeto e implementação de software. O software deve ser produzido para atender às especificações.

3. Validação de software. O software deve ser validado para garantir que atenda às demandas do cliente.

4. Evolução de software. O software deve evoluir para atender às necessidades de mudança dos cliente

Existem também as atividades que dão apoio ao processo, como documentação e gerenciamento de configuração de software

Processo é composto por:

1 - Atividades

2 - Produtos

3 - Papéis

4 - Pré e pós condições

Até aqui 11/03/2025

2.1 Modelos de processo de software

Os modelos de processo:

1 - O modelo em cascata.

2 - Desenvolvimento incremental.

3 - Engenharia de software orientada a reúso.

Esses modelos não são mutuamente exclusivos e muitas vezes são usados em conjunto, especialmente para o desenvolvimento de sistemas de grande porte.

2.2 Atividades do processo

Processos reais de software são intercalados com sequências de atividades técnicas, de colaboração e de gerência, com o intuito de especificar, projetar, implementar e testar um sistema de software.

2.2.1 Especificação de software

Especificação de software ou engenharia de requisitos é o processo de compreensão e definição dos serviços requisitados do sistema e identificação de restrições relativas à operação e ao desenvolvimento do sistema

Existem quatro atividades principais do processo de engenharia de requisitos:

1 Estudo de viabilidade.

2 Elicitação e análise de requisitos.

3 Especificação de requisitos

4 A validação de requisitos

Capítulo 2: Processos de software

SOMMERVILLE, I. Engenharia de software. 9 ed. tradução Ivan Bosnic e Kalinka G.de O. Gonçalves; revisão técnica Kechi Hirama. São Paulo : Pearson Prentice Hall, 2011. (Disponível na Biblioteca da UFGD)

Atividade:

Conteúdo da página 18 até 25

Escolham um dos tipos de processo.

Defina subatividades para a atividade de Especificação de software.

1 - subatividades técnicas, (exemplo: fazer reunião de atribuição de tarefas.)

2 - subatividades de colaboração (exemplo: compartilhar documentação fiscal do cliente); e

3 - subatividades de gerência (exemplo: atualização do cronograma)

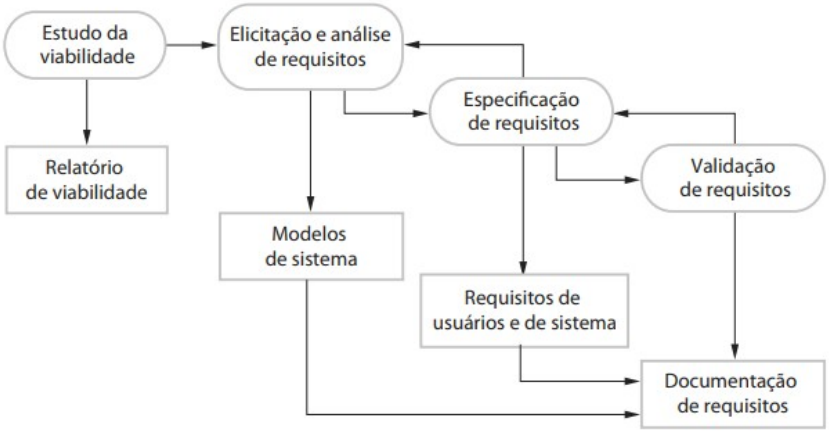
Capítulo 2: Processos de software

SOMMERVILLE, I. Engenharia de software. 9 ed. tradução Ivan Bosnic e Kalinka G.de O. Gonçalves; revisão técnica Kechi Hirama. São Paulo : Pearson Prentice Hall, 2011. (Disponível na Biblioteca da UFGD)

2.2 Atividades do processo (11/03)

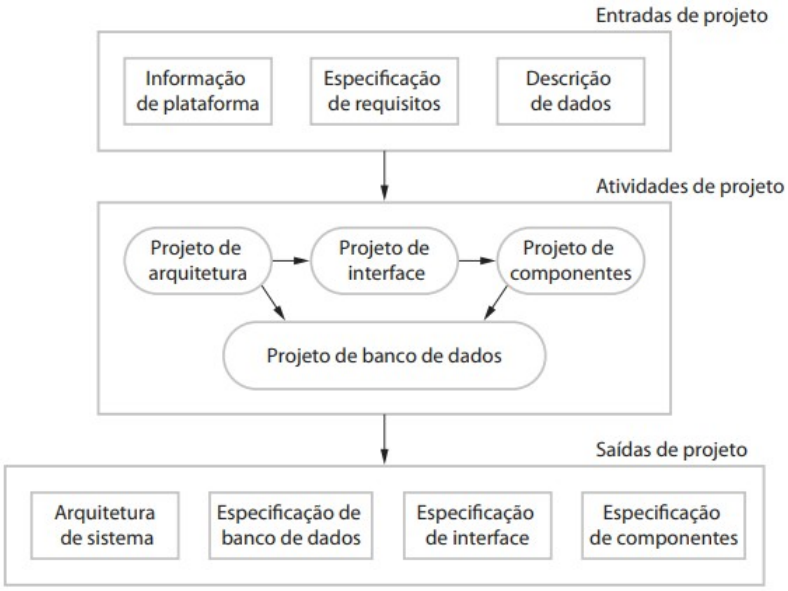
2.2.1 Especificação de software (11/03)

Figura 2.4 Os requisitos da engenharia de processos



2.2.2 Projeto e implementação de software

Figura 2.5 Um modelo geral do processo de projeto



2.2.3 Validação de software

Validação de software ou, mais genericamente, verificação e validação (V&V), tem a intenção de mostrar que um software se adequa a suas especificações ao mesmo tempo que satisfaz as especificações do cliente do sistema

Figura 2.6 Estágios de testes

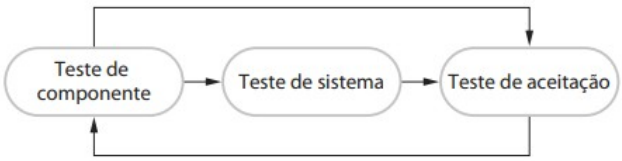
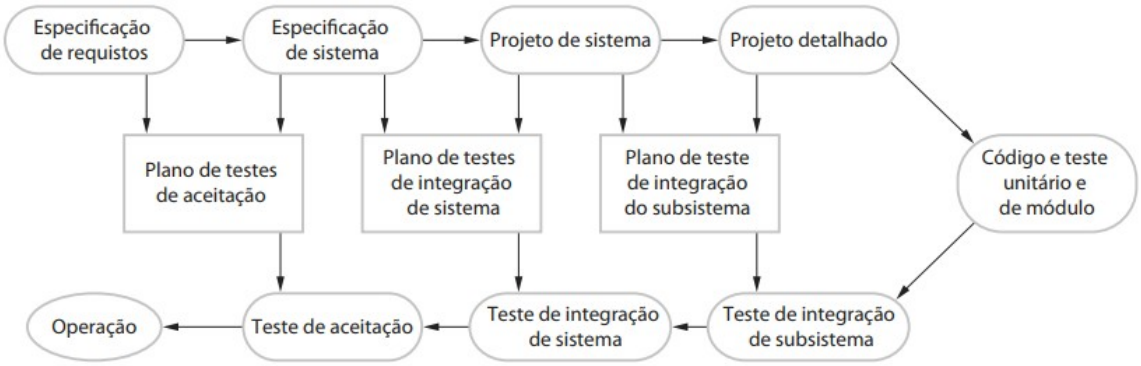
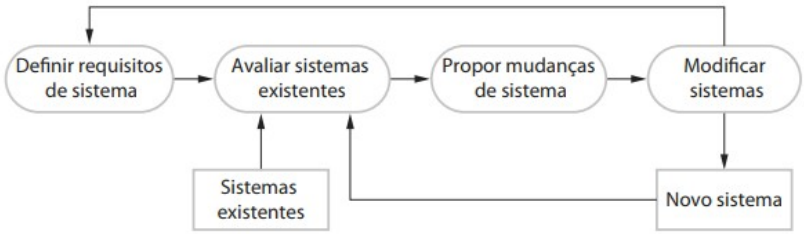


Figura 2.7 Fases de testes de um processo de software dirigido a planos



2.2.4 Evolução do software

Figura 2.8 Evolução do sistema



2.3 Lidando com mudanças

A mudança é inevitável em todos os grandes projetos de software. Os requisitos do sistema mudam, ao mesmo tempo que o negócio que adquiriu o sistema responde a pressões externas e mudam as prioridades de gerenciamento.

A mudança aumenta os custos de desenvolvimento de software, porque geralmente significa que o trabalho deve ser refeito. Isso é chamado retrabalho

… duas abordagens… redução de custos de retrabalho:

1. Prevenção de mudanças

2. Tolerância a mudanças,

2 3.1 Prototipação

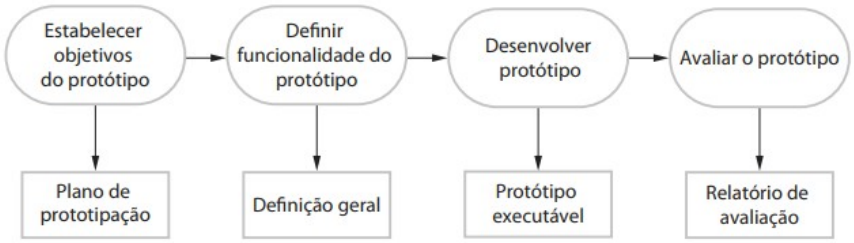
Um protótipo é uma versão inicial de um sistema de software, usado para demonstrar conceitos, experimentar opções de projeto e descobrir mais sobre o problema e suas possíveis soluções.

Um protótipo de software pode ser usado em um processo de desenvolvimento de software

1. No processo de engenharia de requisitos, um protótipo pode ajudar na elicitação e validação de requisitos de sistema.

2. No processo de projeto de sistema, um protótipo pode ser usado para estudar soluções específicas do software e para apoiar o projeto de interface de usuário.

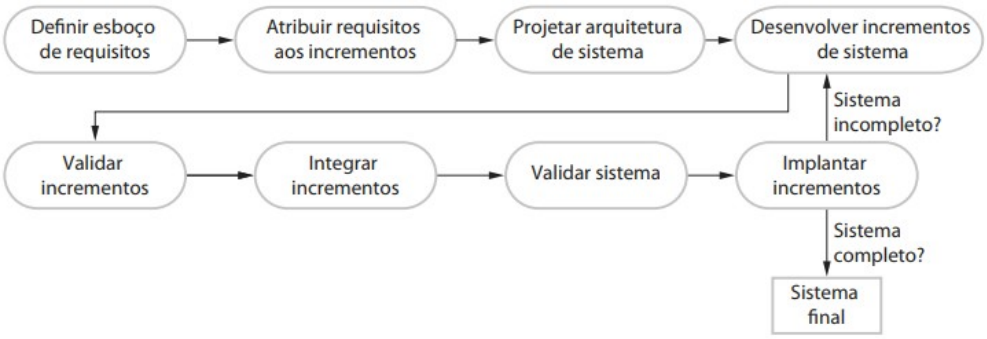
Figura 2.9 O processo de desenvolvimento de protótipo



2.3.2 Entrega incremental

Entrega incremental é uma abordagem para desenvolvimento de software na qual alguns dos incrementos desenvolvidos são entregues ao cliente e implantados para uso em um ambiente operacional.

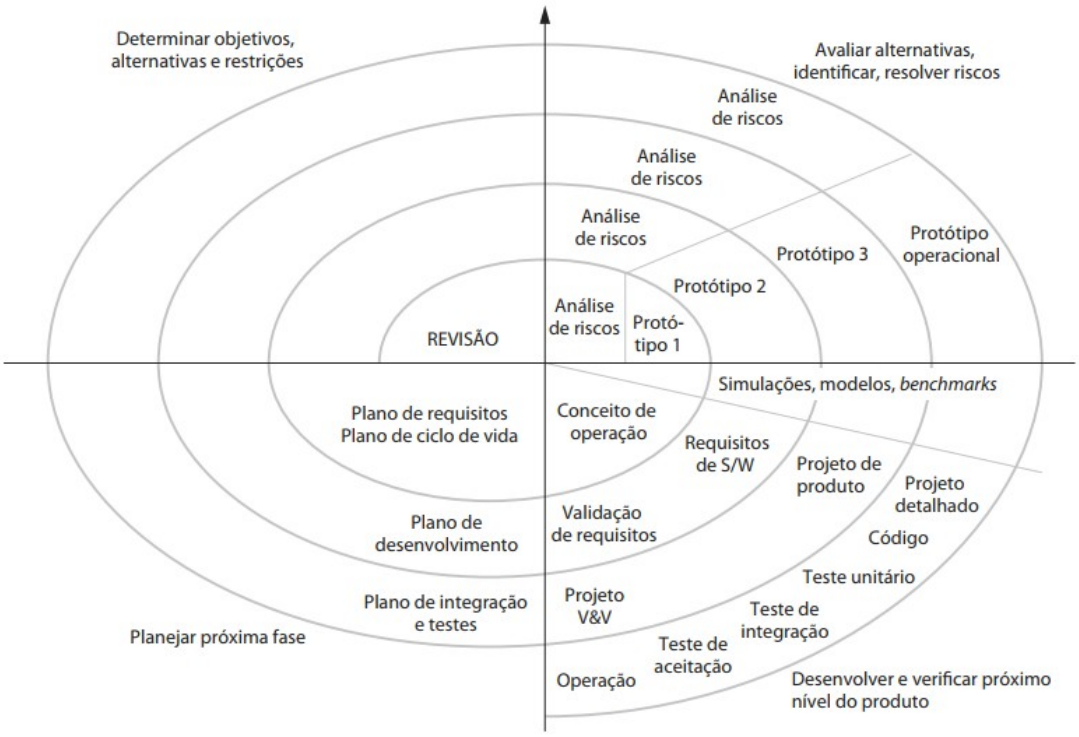
Figura 2.10 Entrega incremental



2.3.3 Modelo espiral de Boehm

Um framework de processo de software dirigido a riscos (o modelo em espiral) foi proposto por Boehm (1988).

Figura 2.11 Modelo em espiral de processo de software de Boehm (©IEEE 1988)



Cada volta na espiral representa uma fase do processo de software.

Dessa forma, a volta mais interna pode preocupar-se com a viabilidade do sistema; o ciclo seguinte, com definição de requisitos; o seguinte, com o projeto do sistema, e assim por diante

A principal diferença entre o modelo espiral e outros modelos de processo de software é seu reconhecimento explícito do risco.

2.4 Rational Unified Process (RUP)

O Rational Unified Process — RUP (KRUTCHEN, 2003) é um exemplo de modelo de processo moderno, derivado de trabalhos sobre a UML e o Unified Software Development Process associado (RUMBAUGH, et al., 1999; ARLOW e NEUSTADT, 2005).

… exemplo de processo híbrido…

Ele reúne elementos de todos os modelos de processo genéricos (Seção 2.1), ilustra boas práticas na especificação e no projeto (Seção 2.2) e apoia a prototipação e a entrega incremental (Seção 2.3).

três perspectivas:

1. Uma perspectiva dinâmica, que mostra as fases do modelo ao longo do tempo.

2. Uma perspectiva estática, que mostra as atividades realizadas no processo.

3. Uma perspectiva prática, que sugere boas práticas a serem usadas durante o processo.

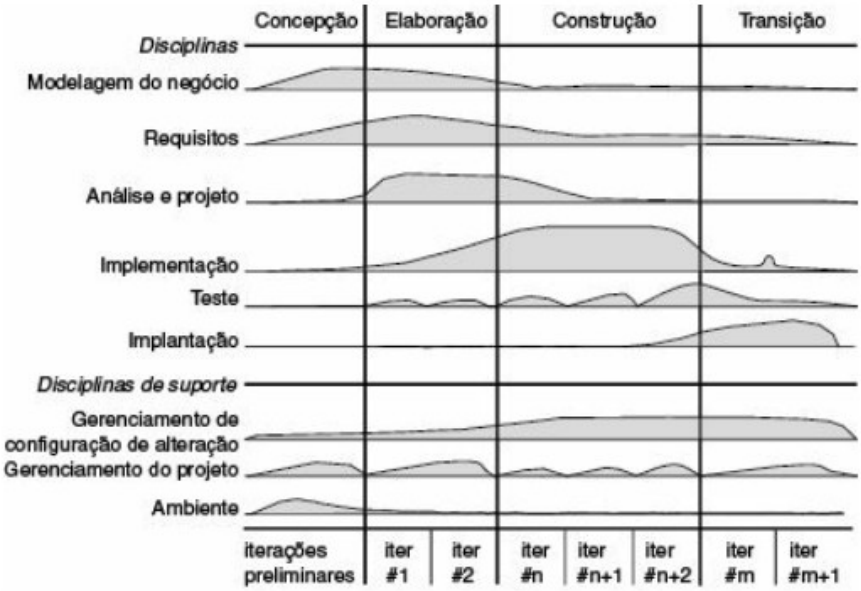


Figura B.1:
O ciclo de vida de desenvolvimento de um software

Capítulo 9: Evolução de software

SOMMERVILLE, I. Engenharia de software. 9 ed. tradução Ivan Bosnic e Kalinka G.de O. Gonçalves; revisão técnica Kechi Hirama. São Paulo : Pearson Prentice Hall, 2011. (Disponível na Biblioteca da UFGD)

INTRODUÇÃO

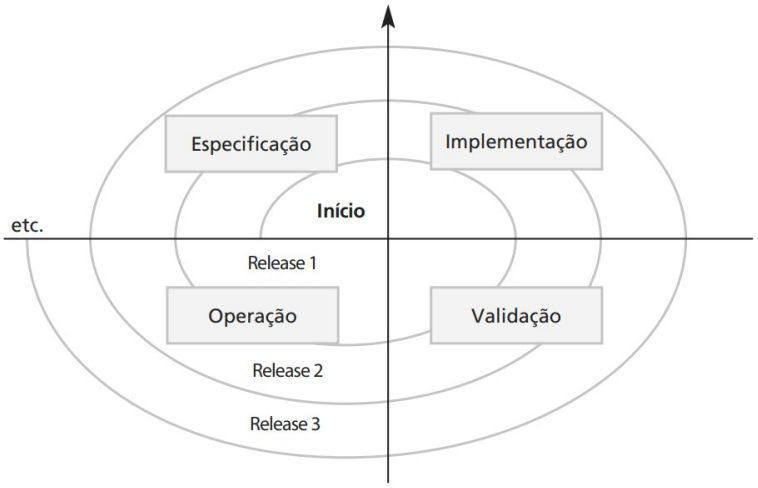
O desenvolvimento de software não é interrompido quando o sistema é entregue, mas continua por toda a vida útil do sistema.

Baseado em uma pesquisa informal da indústria, Erlikh (2000) sugere que 85% a 90% dos custos organizacionais de software são custos de evolução.

Sistemas de software úteis muitas vezes têm uma vida útil muito longa. Por exemplo, sistemas militares ou de infraestrutura de grande porte, como sistemas de controle de tráfego aéreo, podem ter uma vida de 30 anos ou mais.

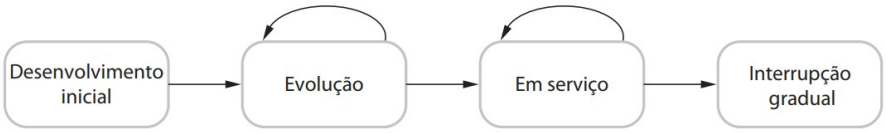
… você deve pensar na engenharia de software como um processo em espiral com requisitos, projeto, implementação e testes que dura toda a vida útil do sistema.

Figura 9.1 Um modelo em espiral de desenvolvimento e evolução



… uma visão alternativa do ciclo de vida de evolução do software. Nesse modelo, eles distinguem ‘evolução’ e ‘em serviço’.

Figura 9.2 Evolução e em serviço



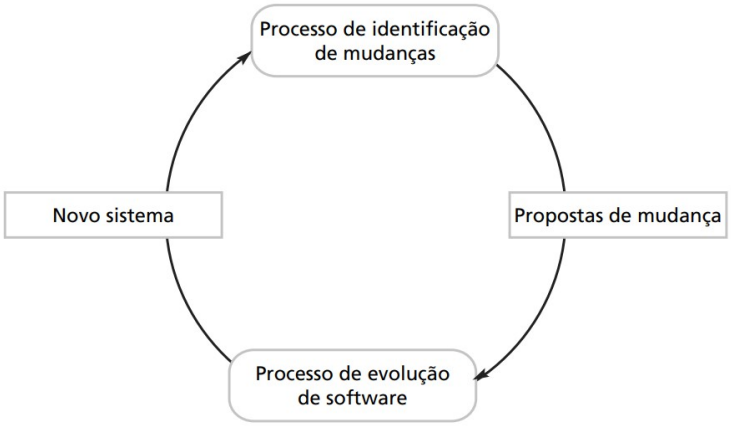
9.1 Processos de evolução

Em algumas organizações, a evolução pode ser um processo informal em que as solicitações de mudança resultam, na maior parte, das conversas entre os usuários do sistema e desenvolvedores.

Em outras empresas, é um processo formal com documentação estruturada produzida em cada estágio do processo.

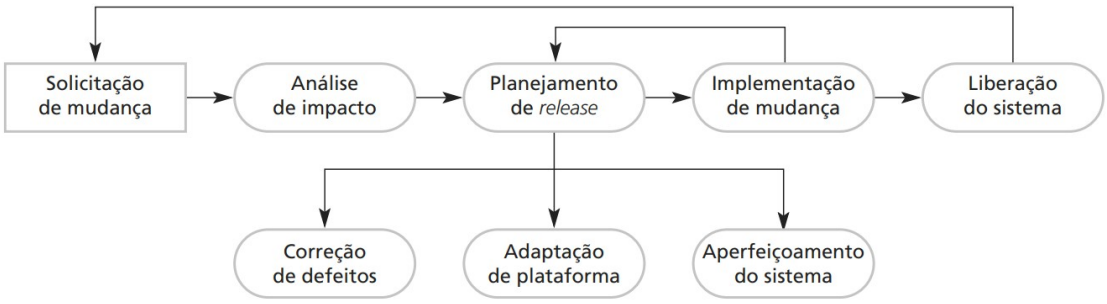
As propostas de mudança podem vir de requisitos já existentes que não tenham sido implementados no release de sistema, solicitações de novos requisitos, relatórios de bugs do sistema apontados pelos stakeholders e novas ideias para melhoria do software vindas da equipe de desenvolvimento.

Figura 9.3 Processos de identificação de mudanças e de evolução



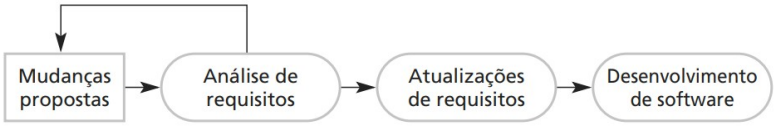
O processo inclui as atividades fundamentais de análise de impacto, planejamento de release, implementação de sistema e liberação de um sistema para os clientes.

Figura 9.4 O processo de evolução de software



Idealmente, o estágio de implementação desse processo deve modificar a especificação, o projeto e a implementação do sistema para refletir as alterações do sistema.

Figura 9.5 Implementação de mudança



… as correções de emergência no sistema devem ser concluídas o mais rapidamente possível. … possivelmente optou por uma solução rápida e viável, e não a melhor solução.

Idealmente, quando são feitas correções de emergência no código, após os defeitos serem corrigidos, a solicitação de mudança deve permanecer em aberto.

9.2 Dinâmica da evolução de programas

Lehman e Belady alegam que essas leis são suscetíveis de serem verdadeiras para todos os tipos de sistemas de software de grandes organizações (que eles chamam sistemas E-type). Nesses sistemas, os requisitos estão mudando para refletir as necessidades dos negócios. Novos releases do sistema são essenciais para o sistema fornecer valor ao negócio

Tabela 9.1 Leis de Lehman

Lei	Descrição
Mudança contínua	Um programa usado em um ambiente do mundo real deve necessariamente mudar, ou se torna progressivamente menos útil nesse ambiente.
Aumento da complexidade	Como um programa em evolução muda, sua estrutura tende a tornar-se mais complexa. Recursos extras devem ser dedicados a preservar e simplicar a estrutura.
Evolução de programa de grande porte	A evolução de programa é um processo de autorregulação. Atributos de sistema como tamanho, tempo entre *releases* e número de erros relatados são aproximadamente invariáveis para cada *release* do sistema.
Estabilidade organizacional	Ao longo da vida de um programa, sua taxa de desenvolvimento é aproximadamente constante e independente dos recursos destinados ao desenvolvimento do sistema.
Conservação da familiaridade	Durante a vigência de um sistema, a mudança incremental em cada *release* é aproximadamente constante.
Crescimento contínuo	A funcionalidade oferecida pelos sistemas tem de aumentar continuamente para manter a satisfação do usuário.
Declínio de qualidade	A qualidade dos sistemas cairá, a menos que eles sejam modificados para refletir mudanças em seu ambiente operacional.
Sistema de *feedback*	Os processos de evolução incorporam sistemas de *feedback* multiagentes, *multiloop*, e você deve tratá-los como sistemas de *feedback* para alcançar significativa melhoria do produto.

Capítulo 9: Evolução de software

SOMMERVILLE, I. Engenharia de software. 9 ed. tradução Ivan Bosnic e Kalinka G.de O. Gonçalves; revisão técnica Kechi Hirama. São Paulo : Pearson Prentice Hall, 2011. (Disponível na Biblioteca da UFGD)

Atividade:

O Capítulo 9 discute a evolução do software, incluindo as mudanças nos requisitos para atender as necessidades dos usuários.

A Figura 9.4, apresenta um diagrama em fluxo do processo de evolução de software. A primeira etapa é “Solicitação de mudança”. Porém, interpretando essa figura, é possível inferir que a solicitação de mudança foi aprovada em entrou no fluxo para ser implementada.

Mas, antes da implementação da “Solicitação de mudanças” essa solicitação deve ser redigida e apresentada para o Engenheiro de Software avaliar se ela será aceita e implementada.

Pense nesse processo de construção da “Solicitação de mudanças”, como o usuário deve proceder para solicitar uma mudança?

Elabore um fluxograma para esse processo.

Capítulo 9: Evolução de software

SOMMERVILLE, I. Engenharia de software. 9 ed. tradução Ivan Bosnic e Kalinka G.de O. Gonçalves; revisão técnica Kechi Hirama. São Paulo : Pearson Prentice Hall, 2011. (Disponível na Biblioteca da UFGD)

Páginas: 174 - 180

9.3 Manutenção de software

… é o processo geral de mudança em um sistema depois que ele é liberado para uso.

O termo geralmente se aplica ao software customizado em que grupos de desenvolvimento separados estão envolvidos antes e depois da liberação.

Existem três diferentes tipos de manutenção de software:

1. Correção de defeitos
2. Adaptação ambiental
3. Adição de funcionalidade.

Figura 9.7 Distribuição do esforço de manutenção

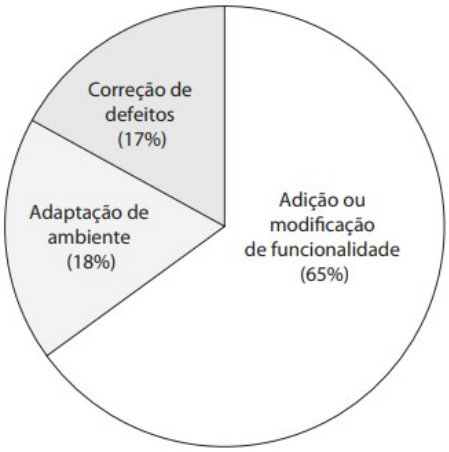
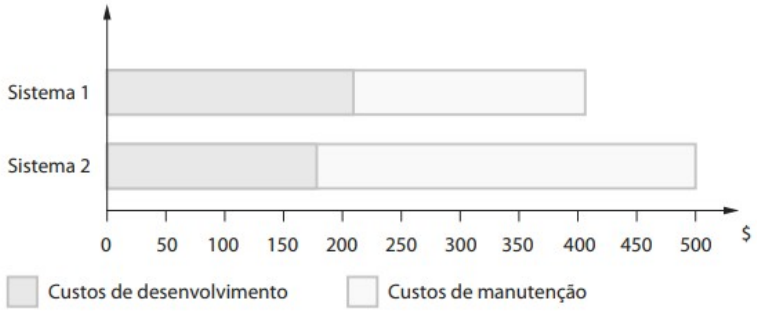


Figura 9.8 mostra como maiores esforços durante o desenvolvimento do sistema para produção de um sistema manutenível reduzem os custos gerais durante a vida útil do sistema

Figura 9.8 Custo de desenvolvimento e manutenção



Geralmente, é mais caro adicionar funcionalidade depois que um sistema está em operação do que implementar a mesma funcionalidade durante o desenvolvimento. As razões para isso são:

1. Estabilidade da equipe.
2. Más práticas de desenvolvimento.
3. Qualificações de pessoal
4. Idade do programa e estrutura.

9.3.1 Previsão de manutenção

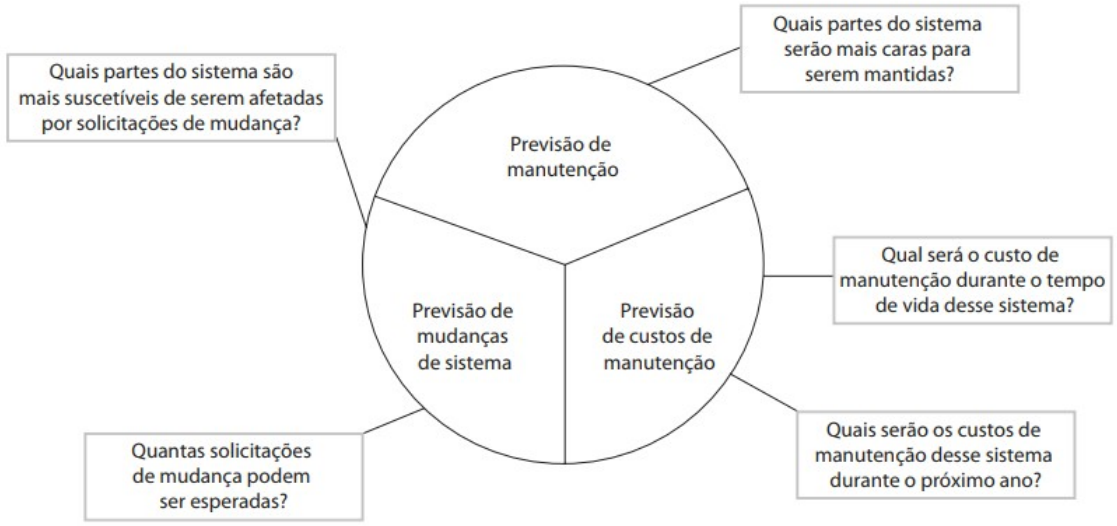
Prever o número de solicitações de mudança para um sistema requer uma compreensão do relacionamento entre o sistema e seu ambiente externo.

Para avaliar os relacionamentos entre um sistema e seu ambiente, você deve avaliar:

1. O número e a complexidade das interfaces de sistema.
2. O número de requisitos inerentemente voláteis de sistema

3. Os processos de negócio em que o sistema é usado.

Figura 9.9 Previsão de manutenção



Depois que um sistema foi colocado em serviço, você pode ser capaz de usar dados de processo para ajudar a prever a manutenibilidade. Exemplos de métricas de processo que podem ser usadas para avaliação de manutenibilidade são apresentados a seguir:

1. Número de solicitações de manutenção corretiva
2. O tempo médio necessário para a análise de impacto.
3. O tempo médio gasto para implementar uma solicitação de mudança.
4. Número de solicitações de mudança pendentes.

9.3.2 Reengenharia de software

muitos sistemas, especialmente sistemas legados mais velhos, são difíceis de serem compreendidos e mudados. Os programas podem ter sido otimizados para o desempenho ou uso de espaço à custa de inteligibilidade, ou, ao longo do tempo, a estrutura inicial do programa pode ter sido danificada por uma série de mudanças.

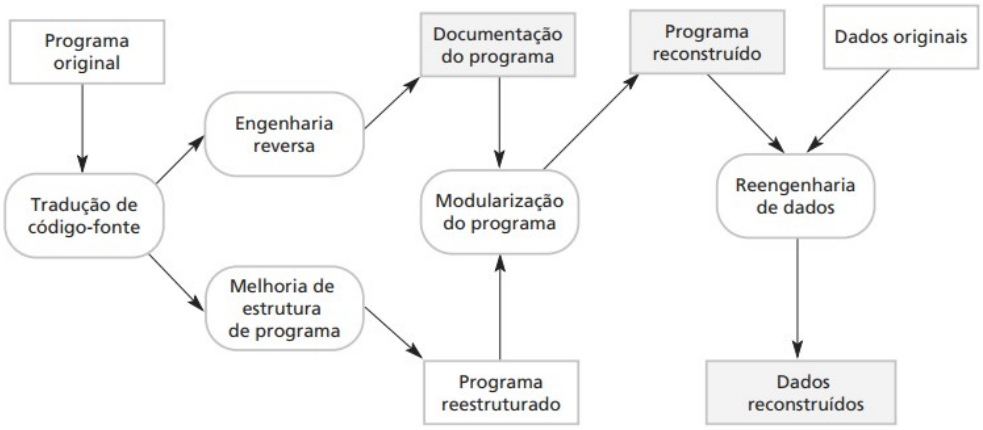
A reengenharia pode envolver a redocumentação de sistema, a refatoração da arquitetura de sistema, a mudança de linguagem de programação para uma linguagem moderna e modificações e atualizações da estrutura e dos dados de sistema.

A funcionalidade de software não é alterada, e você geralmente deve evitar grandes mudanças na arquitetura de sistema.

Existem dois benefícios importantes na reengenharia, em vez de substituição:

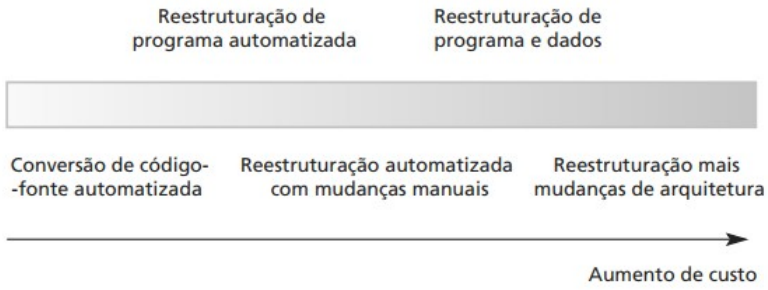
1. Risco reduzido.
2. Custo reduzido.

Figura 9.10 O processo de reengenharia



Os custos da reengenharia, obviamente, dependem da extensão do trabalho.

Figura 9.11 Abordagens de reengenharia



9.3.3 Manutenção preventiva por refatoração

A refatoração é o processo de fazer melhorias em um programa para diminuir a degradação gradual resultante das mudanças (OPDYKE e JOHNSON, 1990)

Isso significa modificar um programa para melhorar sua estrutura, para reduzir sua complexidade ou para torná-lo mais compreensível.

Embora tanto a reengenharia como a refatoração sejam destinadas a tornar o software mais fácil de entender e mudar, elas não são a mesma coisa.

A reengenharia ocorre depois que um sistema foi mantido por algum tempo e com o aumento dos custos de manutenção.

A refatoração é um processo contínuo de melhoria ao longo do processo de desenvolvimento e evolução, com o intuito de evitar a degradação do código, que aumenta os custos e as dificuldades de manutenção de um sistema.

9.4 Gerenciamento de sistemas legados

Mais e mais empresas estão começando a entender que o processo de desenvolvimento de um

sistema é um processo integral de ciclo de vida e que uma separação artificial entre desenvolvimento de software e manutenção de software é inútil. No entanto, ainda existem muitos sistemas legados que são sistemas críticos de negócio. Eles precisam ser ampliados e adaptados às mudanças das práticas de e-business.

… decidir sobre a estratégia mais adequada para a evolução desses sistemas. Existem quatro opções estratégicas:

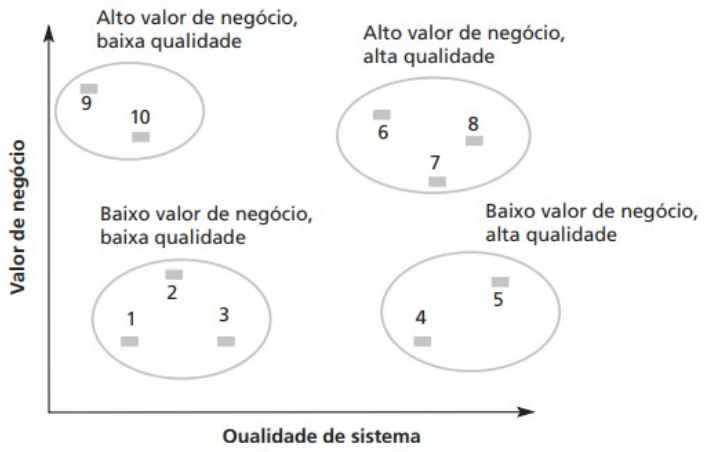
1. Descartar completamente o sistema.
2. Deixar o sistema inalterado e continuar com a manutenção regular
3. Reestruturar o sistema para melhorar sua manutenibilidade.
4. Substituir a totalidade ou parte do sistema por um novo sistema

Quando você está avaliando um sistema legado, precisa olhar para isso de uma perspectiva de negócio e de uma perspectiva técnica (WARREN, 1998).

Por exemplo, suponha que uma organização tenha dez sistemas legados. Você deve avaliar a qualidade e o valor de negócio de cada um e, então, pode criar um gráfico mostrando o valor de negócio relativo e qualidade de sistema, conforme a Figura 9.12.

A partir da Figura 9.12, você pode ver que existem quatro grupos de sistemas:

Figura 9.12 Um exemplo de uma avaliação de sistema legado



Baixa qualidade, baixo valor de negócio.

1. Baixa qualidade, alto valor de negócio.
2. Alta qualidade, baixo valor de negócio.
3. Alta qualidade, alto valor de negócio.

Para avaliar o valor de negócio de um sistema, … quatro questões básicas que você precisa discutir:

1. O uso do sistema
2. Os processos de negócios que são apoiados.
3. Confiança do sistema.
4. As saídas do sistema.

Fatores que você deve considerar na avaliação do ambiente são mostrados na Tabela 9.2.

Tabela 9.2 Fatores usados na avaliação de ambientes

Fator	Questões
Estabilidade do fornecedor	O fornecedor ainda existe? O fornecedor é financeiramente estável e deve continuar existindo? Se o fornecedor não está mais no negócio, existe alguém que mantém os sistemas?
Taxa de falhas	O hardware tem uma grande taxa de falhas reportadas? O software de apoio trava e força o reinício do sistema?
Idade	Quantos anos têm o hardware e o software? Quanto mais velho o hardware e o software de apoio, mais obsoletos serão. Ainda podem funcionar corretamente, mas poderia haver significativos benefícios econômicos e empresariais se migrassem para um sistema mais moderno.
Desempenho	O desempenho do sistema é adequado? Os problemas de desempenho têm um efeito significativo sobre os usuários do sistema?
Requisitos de apoio	Qual apoio local é requisitado pelo hardware e pelo software? Se houver altos custos associados a esse apoio, pode valer a pena considerar a subsituição do sistema.
Custos de manutenção	Quais são os custos de manutenção de hardware e de licenças de software de apoio? Os hardwares mais antigos podem ter custos de manutenção mais elevados do que os sistemas modernos. Os softwares de apoio podem ter altos custos de licenciamento anual.
Interoperabilidade	Existem problemas de interface do sistema com outros sistemas? Compiladores podem, por exemplo, ser usados com as versões atuais do sistema operacional? É necessária a emulação do hardware?

Para avaliar a qualidade técnica de um sistema de aplicação, você precisa avaliar uma série de fatores (Tabela 9.3),

Tabela 9.3 Fatores usados na avaliação de aplicações

Fatores	Questões
Inteligibilidade	Quão difícil é compreender o código-fonte do sistema atual? Quão complexas são as estruturas de controle usadas? As variáveis têm nomes significativos que refletem sua função?
Documentação	Qual documentação do sistema está disponível? A documentação é completa, consistente e atual?
Dados	Existe um modelo de dados explícito para o sistema? Até que ponto os dados nos arquivos estão duplicados? Os dados usados pelo sistema são atuais e consistentes?
Desempenho	O desempenho da aplicação é adequado? Os problemas de desempenho têm um efeito significativo sobre os usuários do sistema?
Linguagem de programação	Compiladores modernos estão disponíveis para a linguagem de programação usada para desenvolver o sistema? A linguagem de programação ainda é usada para o desenvolvimento do novo sistema?
Gerenciamento de configuração	Todas as versões de todas as partes do sistema são gerenciadas por um sistema de gerenciamento de configuração? Existe uma descrição explícita das versões de componentes usadas no sistema atual?
Dados de teste	Existem dados de teste para o sistema? Existem registros dos testes de regressão feitos quando novos recursos forem adicionados ao sistema?
Habilidades de pessoal	Existem pessoas disponíveis com as habilidades necessárias para manter a aplicação? Existem pessoas disponíveis que tenham experiência no sistema?

Dados que podem ser úteis na avaliação de qualidade são:

1. O número de solicitações de mudança no sistema.
2. O número de interfaces de usuário.
3. O volume de dados usados pelo sistema.

Capítulo 9: Evolução de software

SOMMERVILLE, I. Engenharia de software. 9 ed. tradução Ivan Bosnic e Kalinka G.de O. Gonçalves; revisão técnica Kechi Hirama. São Paulo : Pearson Prentice Hall, 2011. (Disponível na Biblioteca da UFGD)

9.3.2 Reengenharia de software

A reengenharia pode envolver a redocumentação de sistema, a refatoração da arquitetura de sistema, a mudança de linguagem de programação para uma linguagem moderna e modificações e atualizações da estrutura e dos dados de sistema. pag 174

Faça uma pesquisa na internet sobre ferramentas CASE (Computer-Aided Software Engineering - ferramentas de computador que auxiliam o desenvolvimento de software), DUAS OU TRÊS, para:

1. Reengenharia
2. Engenharia reversa
3. Reengenharia de dados
4. Refatoração.

Capítulo 10: Sistemas Sociotécnicos

SOMMERVILLE, I. Engenharia de software. 9 ed. tradução Ivan Bosnic e Kalinka G.de O. Gonçalves; revisão técnica Kechi Hirama. São Paulo : Pearson Prentice Hall, 2011. (Disponível na Biblioteca da UFGD)

Introdução

Este capítulo aborda os sistemas sociotécnicos, destacando sua complexidade, interdependência entre componentes técnicos e humanos, e a necessidade de uma abordagem holística para garantir confiabilidade e alinhamento com objetivos organizacionais. A seguir estão os principais conceitos discutidos:

10.1 Sistemas complexos

1. Sistemas Sociotécnicos

• Definição: Sistemas que combinam componentes técnicos (hardware e software) com elementos humanos, sociais e organizacionais.

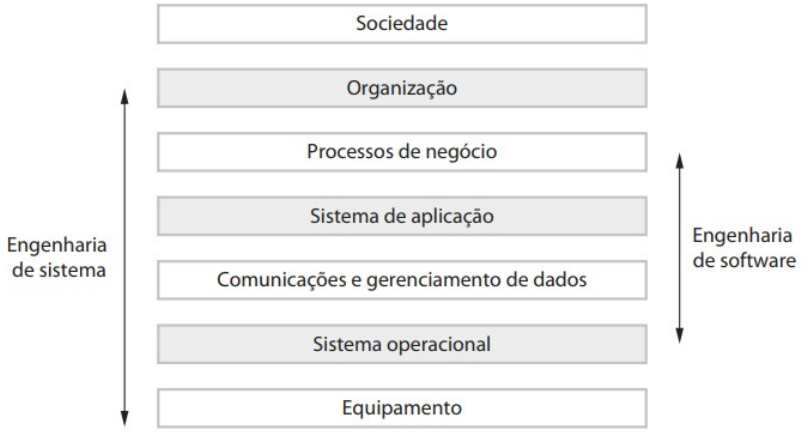
• Exemplo: Um sistema meteorológico que integra software, hardware, processos de previsão, operadores humanos e organizações que utilizam as previsões.

Sistemas Sociotécnicos

Esses sistemas são caracterizados pela interdependência entre seus elementos, exigindo uma abordagem integrada para seu desenvolvimento e operação.

Sistemas sociotécnicos são tão complexos que é praticamente impossível entendê-los como um todo. Em vez disso, você deve percebê-los como camadas

Figura 10.1 A pilha de sistemas sociotécnicos



10.2 Sistemas Complexos

Um sistema é uma coleção intencional de componentes inter-relacionados, de diferentes tipos, que funcionam em conjunto para atingir um objetivo.

Sistemas que incluem software encaixam-se em duas categorias:

1. Sistemas técnicos baseados em computador.

2. Sistemas sociotécnicos.

Os fatores organizacionais do ambiente do sistema que podem afetar os requisitos, projeto e operação de um sistema sociotécnico incluem:

1. Mudanças de processos.

2. Mudanças de trabalho.

3. Mudanças organizacionais.

10.1.1 Propriedades Emergentes

Características do sistema como um todo que não podem ser atribuídas a nenhum componente isolado.

Tabela 10.1 Exemplos de propriedades emergentes

Propriedade	Descrição
Volume	O volume de um sistema (o espaço total ocupado) varia conforme os conjuntos de componentes estão dispostos e conectados.
Confiabilidade	A confiabilidade de sistema depende da confiabilidade de componentes, mas interações inesperadas podem causar novos tipos de falhas e, portanto, afetar a confiabilidade do sistema.
Proteção	A proteção do sistema (sua capacidade de resistir ao ataque) é uma propriedade complexa que não pode ser facilmente mensurada. Os ataques podem ser criados de forma imprevista pelos projetistas de sistemas e, assim, derrotar as proteções internas.
Reparabilidade	Essa propriedade reflete quão fácil é corrigir um problema com o sistema uma vez que este tenha sido descoberto. Depende da capacidade de diagnosticar o problema e do acesso a componentes que estejam com defeito, bem como de se modificar ou substituir tais componentes.
Usabilidade	Essa propriedade reflete quão fácil é usar o sistema. Depende dos componentes técnicos de sistema, seus operadores e seu ambiente operacional.

Essas propriedades só podem ser avaliadas no nível do sistema completo.

Sistemas sociotécnicos são parcialmente não determinísticos, porque incluem pessoas e parcialmente porque as alterações de hardware, software e dados nesses sistemas são muito frequentes.

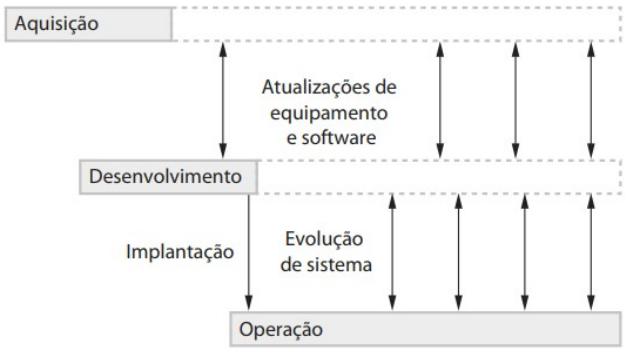
Um sistema determinístico é aquele que é totalmente previsível. Se ignorarmos problemas de timing, os sistemas de software que rodam em hardwares totalmente confiáveis quando apresentados a uma sequência de entradas produzirão sempre a mesma sequência de saídas.

10.2 Engenharia de Sistemas

A Engenharia de Sistemas é um campo interdisciplinar que trata do desenvolvimento e gerenciamento de sistemas sociotécnicos complexos. Ela abrange tanto os aspectos técnicos quanto os organizacionais, garantindo que os sistemas atendam aos requisitos funcionais e não funcionais.

É uma abordagem abrangente para especificar, projetar, implementar e operar sistemas complexos. Inclui a integração de hardware, software, processos organizacionais e humanos.

Figura 10.3 Estágios da engenharia de sistemas



Atividade Prática: Especificação de um Sistema Sociotécnico

Objetivo:

Especificação de um sistema sociotécnico para um problema real, considerando tanto os aspectos técnicos quanto os sociais.

1- Identificação do Problema

- Em grupos de 4 a 5 pessoas.
- Identifiquem um problema real em sua comunidade ou no ambiente de trabalho que possa ser resolvido com um sistema sociotécnico. Façam uma descrição consistente do problema (identificar e contextualizar o problema e analisar as características do problema).
- Exemplos: gestão de resíduos, monitoramento de saúde, gestão de trânsito, etc.

2. Brainstorming

- Registre em um arquivo texto sobre como o problema pode ser abordado, considerando os aspectos de hardware, software, processos organizacionais e humanos.

Capítulo 10: Sistemas Sociotécnicos

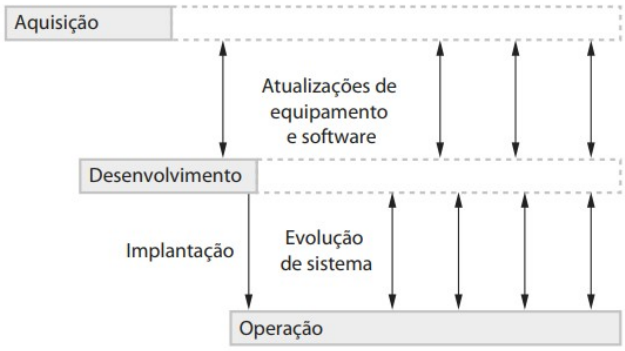
SOMMERVILLE, I. Engenharia de software. 9 ed. tradução Ivan Bosnic e Kalinka G.de O. Gonçalves; revisão técnica Kechi Hirama. São Paulo : Pearson Prentice Hall, 2011. (Disponível na Biblioteca da UFGD)

10.2 Engenharia de Sistemas - faltou na aula passada

A Engenharia de Sistemas é um campo interdisciplinar que trata do desenvolvimento e gerenciamento de sistemas sociotécnicos complexos. Ela abrange tanto os aspectos técnicos quanto os organizacionais, garantindo que os sistemas atendam aos requisitos funcionais e não funcionais.

É uma abordagem abrangente para especificar, projetar, implementar e operar sistemas complexos. Inclui a integração de hardware, software, processos organizacionais e humanos.

Figura 10.3 Estágios da engenharia de sistemas



10.3 Aquisição de sistemas

A aquisição de sistemas refere-se ao processo de obtenção de um sistema completo, que pode incluir hardware, software e serviços associados. Esse processo é particularmente relevante em sistemas sociotécnicos devido à sua complexidade e à necessidade de integração entre diferentes componentes.

Aspectos Principais da Aquisição

Compra versus Desenvolvimento Interno:

- A decisão entre adquirir um sistema pronto (off-the-shelf) ou desenvolver um sistema personalizado depende de fatores como custo, tempo, requisitos específicos e disponibilidade de soluções no mercado.

Requisitos e Especificações:

- A definição clara dos requisitos é essencial para garantir que o sistema adquirido atenda às necessidades organizacionais.
- Muitas vezes, os requisitos precisam ser adaptados para se alinhar às capacidades do sistema disponível.

Integração com Sistemas Existentes:

- Os sistemas adquiridos geralmente precisam ser integrados a outros sistemas já operacionais na organização, o que pode exigir adaptações técnicas significativas.

Contratos e Fornecedores:

- A relação com fornecedores é crítica, especialmente em termos de contratos que definem responsabilidades, suporte técnico e garantias.

Riscos Associados:

- A aquisição pode envolver riscos relacionados à dependência de fornecedores, falhas na integração ou incompatibilidade com processos organizacionais.

Fases da Aquisição

- Planejamento: Identificação das necessidades e análise das opções disponíveis.

- Seleção: Avaliação e escolha do fornecedor ou solução mais adequada.

- Implementação: Instalação, configuração e integração do sistema.

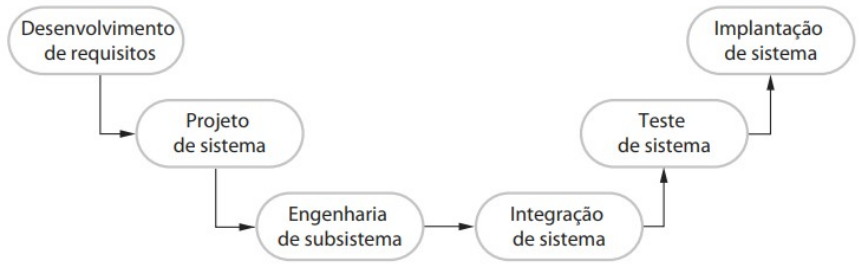
-Operação: Uso contínuo do sistema com suporte técnico conforme necessário.

A aquisição de sistemas é uma atividade estratégica em engenharia de software, especialmente em grandes organizações ou projetos governamentais, onde a escolha entre desenvolver internamente ou adquirir externamente pode impactar significativamente os custos, prazos e resultados finais do projeto.

10.4 Desenvolvimento de sistemas

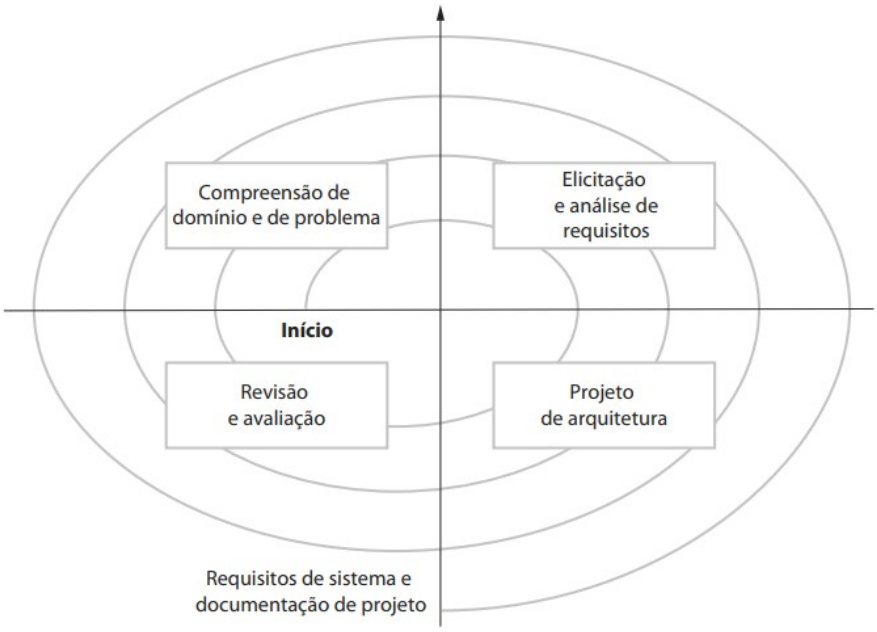
Os objetivos do processo de desenvolvimento do sistema são desenvolver ou adquirir todos os componentes de um sistema e, em seguida, integrar esses componentes para criar um sistema final. Os requisitos são a ponte entre os processos de aquisição e de desenvolvimento. Durante a aquisição, requisitos de negócio e requisitos de sistema funcionais e não funcionais de alto nível são definidos.

Figura 10.6 Desenvolvimento de sistemas



Começando no centro, cada volta da espiral pode adicionar detalhes aos requisitos e ao projeto. Algumas voltas podem se concentrar nos requisitos, e outras, no projeto.

Figura 10.7 Espiral de requisitos e projeto



10.5 Operação de sistemas

Os processos operacionais são os processos envolvidos no uso do sistema para seus fins definidos. Por exemplo, os operadores de um sistema de controle de tráfego aéreo seguem processos específicos quando as aeronaves entram e saem do espaço aéreo, quando precisam alterar a altura ou a velocidade, quando ocorre uma emergência, e assim por diante. Para novos sistemas, esses processos operacionais precisam ser definidos e documentados durante o processo de desenvolvimento do sistema.

10.5.1 Erro humano

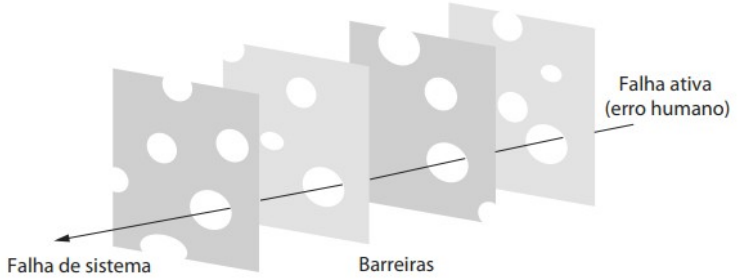
Às vezes, as pessoas cometem erros no uso do sistema, e estes podem causar a falha do sistema.

Existem duas maneiras de ver esse problema:

1. A abordagem de pessoas.

2. A abordagem de sistemas.

Figura 10.8 Modelo queijo suíço de Reason de falha de sistema



10.5.2 Evolução do sistema

Sistemas complexos de grande porte têm vida útil muito longa.

Durante sua vida, eles são alterados para:

1. Correção dos erros nos requisitos do sistema original e para implementar novos requisitos.
2. Os computadores do sistema tendem a ser substituídos por máquinas novas e mais rápidas.
3. A organização que usa o sistema pode reorganizar-se e, portanto, usar o sistema de uma maneira diferente.
4. O ambiente externo do sistema pode mudar ou forçar mudanças

Capítulo 10: Sistemas Sociotécnicos

SOMMERVILLE, I. Engenharia de software. 9 ed. tradução Ivan Bosnic e Kalinka G.de O. Gonçalves; revisão técnica Kechi Hirama. São Paulo : Pearson Prentice Hall, 2011. (Disponível na Biblioteca da UFGD)

Continuação - Atividade Prática: Especificação de um Sistema Sociotécnico

3 - Especificação do Sistema

• Desenvolva uma especificação básica do sistema que propuseram, utilizando técnicas de especificação formal ou informal.

• Considerem:

- Componentes Técnicos: Tecnologias necessárias, infraestrutura, etc.

- Componentes Sociais: Impacto nas pessoas, mudanças organizacionais, etc.
- Interoperabilidade: Como os componentes técnicos e sociais interagem.

Elabore um texto descrevendo a especificação.

Capítulo 1, 2, 9 e 10:

SOMMERVILLE, I. Engenharia de software. 9 ed. tradução Ivan Bosnic e Kalinka G.de O. Gonçalves; revisão técnica Kechi Hirama. São Paulo : Pearson Prentice Hall, 2011. (Disponível na Biblioteca da UFGD)

Atividade Prática

Capítulos 1 (Introdução), 2 (Processos de Software), 9 (Evolução de Software) e 10 (Sistemas Sociotécnicos)

Contexto

Você faz parte de uma equipe responsável por modernizar um sistema legado de prontuário eletrônico utilizado em um hospital universitário. O sistema foi desenvolvido há mais de 10 anos, apresenta dificuldades de manutenção e integração com novos sistemas, além de desafios para atender às atuais necessidades dos usuários e requisitos regulatórios.

Atividades

1. Análise Inicial (Capítulo 1 – Introdução à Engenharia de Software)

a) Liste e explique os quatro caracteristicas essenciais que o software hospitalar deve possuir, justificando sua importância para o contexto de saúde.

b) Identifique e discuta dois desafios éticos que podem surgir no desenvolvimento ou manutenção deste sistema.

2. Processo de Software (Capítulo 2 – Processos de Software)

a) Escolha um modelo de processo de software (por exemplo, cascata, incremental ou RUP) que você considera mais adequado para a modernização desse sistema. Justifique sua escolha considerando as características do sistema e do ambiente hospitalar.

b) Descreva as principais atividades (estágios ou fases) do processo de software selecionado e como elas seriam aplicadas neste projeto (defina as ações da equipe de desenvolvimento. Exemplo: Atividade de Definição de Requisitos - Ações: Realizar reuniões com os usuários finais; Elaborar

relatórios sobre as reuniões; Elaborar um documento com as definições dos requisitos; Aprovar o documento com as definições dos requisitos, com assinatura do Cliente e do Engenheiro de Software).

3. Evolução e Manutenção (Capítulo 9 – Evolução de Software)

a) Explique o que é manutenção corretiva, adaptativa e evolutiva. Dê um exemplo prático de cada tipo relacionado ao sistema legado do hospital.

b) O hospital deseja integrar o sistema a um novo módulo de telemedicina. Quais desafios da evolução de software podem surgir nesse contexto? Como você recomendaria lidar com eles?

4. Sistemas Sociotécnicos (Capítulo 10 – Sistemas Sociotécnicos)

a) Identifique pelo menos três componentes sociotécnicos envolvidos no sistema de prontuário eletrônico do hospital (por exemplo, pessoas, processos, infraestrutura tecnológica).

b) Explique como mudanças em um desses componentes podem impactar os demais e o sucesso do projeto de modernização.

Instruções:

• Responda cada item de forma clara e objetiva, utilizando exemplos práticos sempre que possível.

• Utilize conceitos e terminologia apresentados nos capítulos indicados do livro de Sommerville.

• Caso necessário, consulte os estudos de caso apresentados no Capítulo 1 para embasar suas respostas.